

A Dual-Core, Two-Wide Superscalar, Out-of-Order RISC-V Processor

RV32IM · speculative execution · private L1 caches kept coherent with MESI
over an AXI4 crossbar · verified with UVM

An illustrated tour of what it does, how it works, and why it is built this way.

Ahmed Inam Chowdhury

Chapter 0: What this is

A processor is a machine that follows orders. The orders are very simple: add two numbers, fetch something from memory, jump somewhere else. And it follows them very fast. Everything a computer does is built out of those small steps.

This document is a tour of one I built from scratch. It starts with the simplest machine that works at all, and ends with two cores running at the same time and agreeing with each other about what is in memory. Each chapter takes one problem, shows the picture, and then says what I actually built and what it cost.

How to read this

Each chapter has two halves. The first half explains the problem and the idea, in plain words, with no background needed. Then there is a marked line across the page. Below that line I get specific: what I built, why I chose it, and what the measurements said.

If you are not an engineer, you can stop at the line every time and still follow the whole story. If you are, you can start there.

Each chapter ends with a number. Not a claim. A measurement, taken from the machine.

from here on, how it was actually built

The machine implements **RV32IM**: the 32-bit RISC-V base integer instruction set, plus the standard extension for multiply and divide. There are two cores; in RISC-V terms each is a **hart**, a hardware thread.

Each core is **two-wide superscalar** and **out-of-order**. *Superscalar* means it can start more than one instruction in the same clock cycle, and two-wide means two. *Out-of-order* means instructions execute whenever their inputs are ready rather than in the order they were written, and are put back into that order before anything they did becomes permanent. It is also **speculative**: it predicts which way branches will go and runs ahead on that assumption, undoing the work when the prediction turns out wrong.

Each core has its own private **level-one caches**, one for instructions and one for data. The two data caches are kept consistent by a hardware **cache-coherence protocol**, specifically **MESI** maintained by **snooping**. Underneath sits an **AXI4 crossbar** joining two bus masters to two slaves: main memory, and a timer and inter-core interrupt unit. The whole thing is

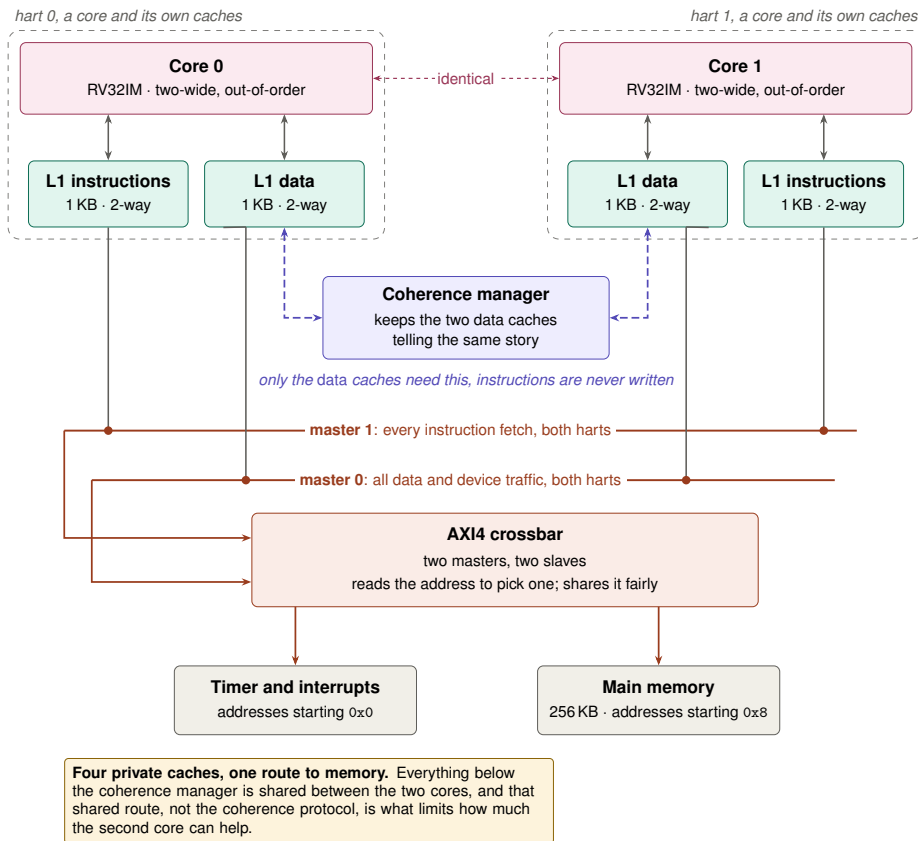


Figure 1: **The finished machine.** Two identical cores. Each has its own small caches, fast private copies of the memory it is using. The two data caches have to agree with each other, which is what the coherence manager is for. Below that, everything is shared: one route to memory, one timer. Every later chapter is a close-up of one box in this picture.

verified with a **UVM** testbench.

It was built in stages, and each stage had to keep every earlier measurement unchanged before it was allowed to continue. That rule caught more mistakes than any other single decision in the project.

The full engineering report is a separate, longer document. Where you want more depth than this gives you, that is where to go.

Chapter 1: One instruction at a time

Start with the simplest thing that works.

A program is a long list of small orders sitting in memory. The machine reads one, works out what it means, does it, and writes down the answer. Then it reads the next one. That is the whole idea.

How it works

There are only a few moving parts. The *register file* is a tiny set of 32 slots holding the values the program is working with right now. The *ALU* is the part that does arithmetic. The *program*

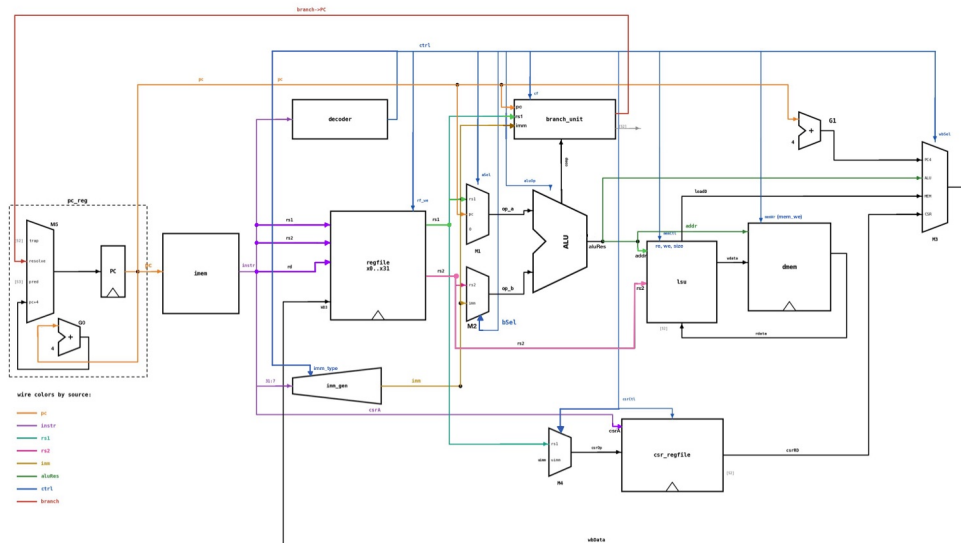


Figure 2: **The simplest working processor.** Left to right: the program counter says which instruction to read; the instruction memory hands it over; the middle works out what it means and pulls the numbers it needs out of the register file; the ALU does the arithmetic; the result goes back into a register or out to memory. All of that happens in one tick of the clock.

counter remembers which instruction we are on.

Every instruction takes the same journey through those parts, and the whole journey finishes before the next instruction starts.

————— *from here on, how it was actually built* —————

Doing everything in one clock tick sounds efficient and is the opposite. The clock has to be slow enough for the *slowest* instruction to finish completely, read the instruction, read registers, do the arithmetic, reach memory, write back. Every fast instruction then waits for that same long tick.

So the machine is correct, simple to reason about, and wasteful. Most of the hardware sits idle most of the time: while the ALU is working, the instruction memory has nothing to do.

That observation is the whole of the next chapter.

The choices, spelled out

- **Separate instruction and data memory.** Instructions come from one memory and data from another, so a load never has to wait for a fetch. This is a deliberate simplification at this stage: it lets execution be got right without also fighting memory timing. Chapter 6 replaces it with split *caches* over one shared memory.
- **Comparison shares the arithmetic unit’s subtractor.** A branch asks “is a equal to b ”, “is a less than b ”. All of those fall out of $a - b$, which the machine can already do. So there is no separate comparator: reuse comes from shared hardware, not from a shared name.
- **Loads extract then extend; stores replicate then mask.** A byte load pulls the

addressed byte out first and only then widens it to 32 bits, which is exactly why signed and unsigned byte loads are different instructions. A byte store does the opposite: it copies the byte across all four lanes and lets a four-bit mask decide which one is really written. That mask is the same idea the memory bus itself uses.

What it cost: One instruction per clock tick, but the tick has to be long enough for the worst case. Nothing is ever busy at the same time as anything else.

Chapter 2: Five things at once

In a restaurant kitchen nobody waits for one dish to be ordered, cooked, plated and served before starting the next. While one plate is being garnished, the next is on the stove, and a third order is still being written down. Each station does one job and passes the dish along.

The machine is the same. While one instruction is doing arithmetic, the next can be read from memory.

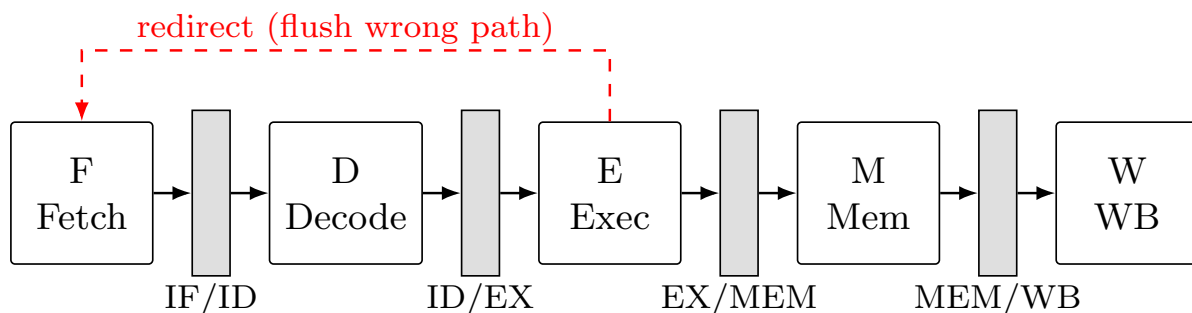


Figure 3: **Five stages, five instructions in flight.** Fetch reads the instruction. Decode works out what it means and reads the registers. Execute does the arithmetic. Memory reads or writes data. Writeback saves the result. The grey bars between stages are latches, small holding pens that let each stage keep its own instruction while the others work on different ones.

How it works

Split the journey into five stages and let five different instructions occupy them at once. The clock tick now only has to be long enough for the slowest *stage*, not the whole journey, so it gets much shorter.

Once the pipeline is full, one instruction finishes every tick. Each individual instruction still takes five ticks to get through. Pipelining does not make any one instruction faster. It makes more of them finish per second, which is what you actually care about.

from here on, how it was actually built

This is the classic **five-stage RISC pipeline**: instruction fetch (IF), instruction decode and register read (ID), execute (EX), memory access (MEM) and write-back (WB). The latches between stages are called **pipeline registers**. Everything a stage needs travels with it through those registers, so no stage ever has to reach backwards for information.

Two words worth separating. **Latency** is how long one instruction takes end to end, and

pipelining makes it slightly worse. **Throughput** is how many finish per unit time, and pipelining is entirely about that. The usual measure is **IPC**, instructions per cycle.

On a branch-heavy test program this machine ran at 0.82 instructions per tick. That is good, but it is not 1.0, and the missing part is not rounding. It is a real cost with a real cause, and the next chapter is entirely about it.

The choices, spelled out

- **Five stages: fetch, decode, execute, memory, writeback.** With four sets of latches between them. Everything an instruction needs travels with it, so no stage ever reaches backwards for information.
- **A register file that reads what is being written this cycle.** If an instruction is writing a register in the same tick another is reading it, the read returns the new value, not the old one. Without this the machine silently gets wrong answers only when a dependency is exactly three instructions apart, which is rare enough to pass every short test and then fail on real code.
- **One redirect mux, with a fixed priority.** Trap beats branch resolution, which beats prediction, which beats simply carrying on. Later stages wire into the spare inputs of that mux rather than rebuilding it.

What it cost: 0.82 instructions per clock tick, measured on a branch-heavy program. The gap below 1.0 is the subject of chapter 3.

Chapter 3: When the trick breaks

Overlapping only works if the jobs are independent. Laundry loads are. Program instructions very often are not.

If instruction two needs the answer from instruction one, and instruction one has not finished yet, then instruction two cannot simply carry on.

These situations are called **hazards**, and they come in three kinds. A **data hazard** is one instruction needing a value another has not produced yet. A **control hazard** is not knowing which instruction comes next, because a branch has not been decided. A **structural hazard** is two instructions wanting the same piece of hardware at once. This pipeline sidesteps that one by giving instruction fetch and data access separate paths, so nothing ever queues for a port. That reprieve is temporary, and it is worth knowing now that it does not last. Going **two-wide** means two instructions want a functional unit, a register-file port and a memory port in the same cycle. Going **out of order** means they need not even be adjacent instructions. Structural hazards come back in force at both, and chapter 7 shows the three different ways this design deals with each one. Adding a second core brings a further set again, because everything below the caches is shared between them, and chapter 8 measures what that costs. The three figures below are the first two kinds.

How it works

Three problems, three answers, and each has a name.

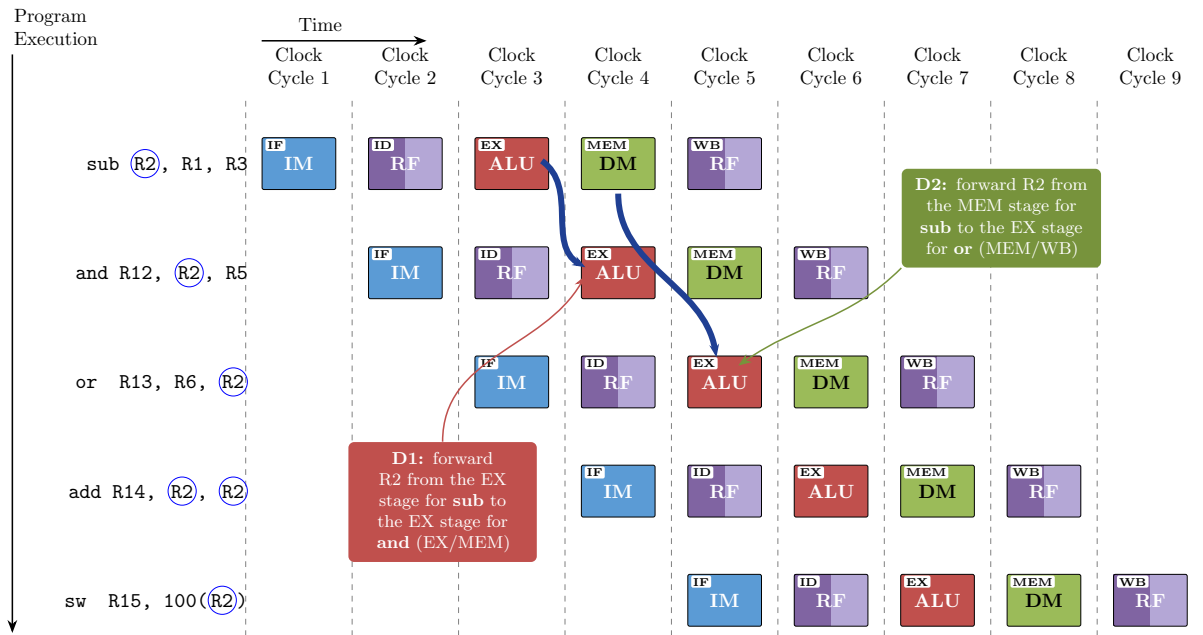
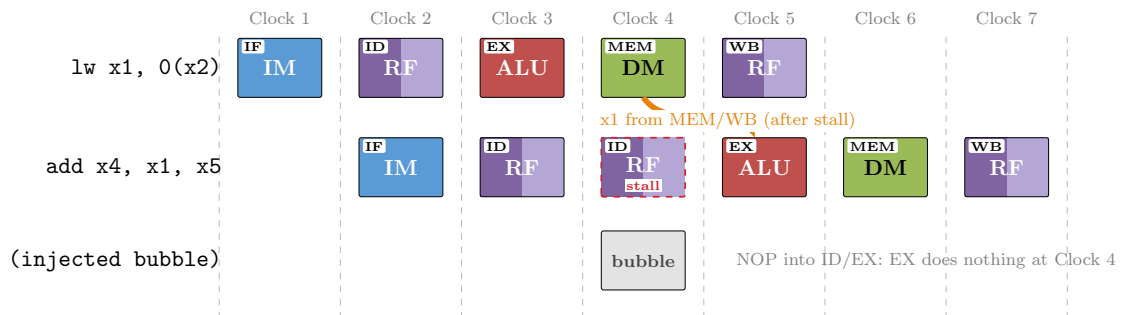


Figure 4: **Case one: the answer exists, it is just not filed yet.** The first instruction computes a value that the next two need. It has not written it back to the register file yet, but the value is physically sitting in a latch. So a wire carries it straight to where it is needed (the thick arrows). Nothing waits. This is called forwarding, and it solves the common case completely.

D5: load-use: STALL one cycle, then forward

Load data is born at END of MEM; the consumer needs it at START of EX. Forwarding alone cannot serve it.



D5c: load feeds store DATA: forward at MEM, NO stall

x1 is store data (rs2), used only in MEM (not EX). Forward load → store at MEM. No stall (your policy).

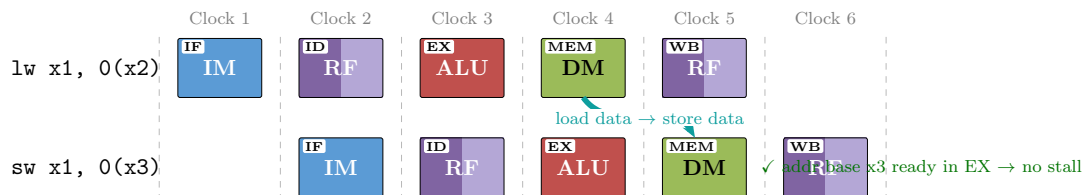
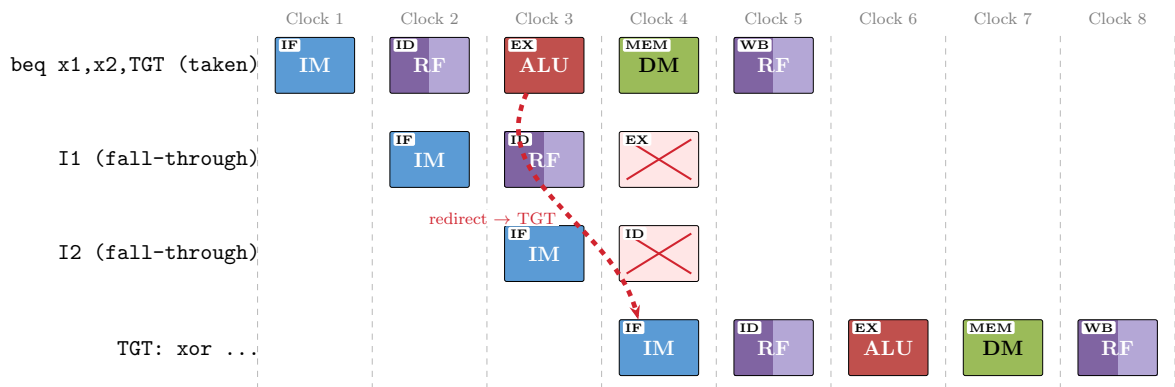


Figure 5: **Case two: forwarding is not enough.** A load gets its value from memory, and memory answers at the *end* of the memory stage, one tick after the instruction that needs it has already started its arithmetic. There is no wire that can carry a value backwards in time. The only answer is to stop the pipeline for one tick and let the value catch up. This is the case that costs something no matter how clever the wiring is.

C1: taken branch: flush 2

beq resolves TAKEN in EX. The two fall-through instrs already fetched are wrong-path. Penalty 2.



C2: not-taken branch: NO flush

Branch NOT taken: the sequentially-fetched instruction is the correct path. Nothing killed. Free.

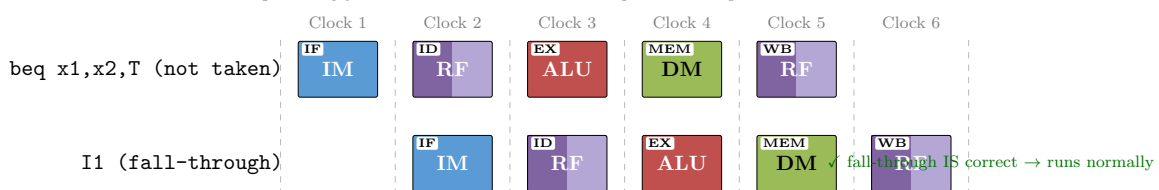


Figure 6: **Case three: we went the wrong way.** A branch decides where to go next, but by the time it decides, the machine has already fetched two more instructions from the road not taken. Those two are cancelled (the crosses) and fetching restarts from the right place. The work is simply lost. This is what chapter 4 exists to make rare.

If the value already exists somewhere in the pipeline, wire it directly to where it is needed. That is **forwarding**, also called **bypassing**.

If it genuinely does not exist yet, stop and wait. That is a **stall**, or **pipeline interlock**, and the empty slot it pushes down the pipeline is a **bubble**. The specific case here, a load feeding the instruction immediately after it, is the **load-use hazard**.

If the machine went down the wrong path, throw that work away and restart. That is a **flush**, or **squash**, and the discarded instructions were **speculative**: fetched on an assumption that turned out to be false.

— from here on, how it was actually built —

Forwarding covers the ordinary case at no cost. The load-use case costs exactly one tick and cannot be avoided by wiring; it is a limit of the order the stages happen in.

The branch case is the expensive one, because the amount thrown away grows with how far ahead the machine has run. In this five-stage pipeline it is two instructions. In the out-of-order machine of chapter 7 it can be dozens, and that is why chapter 4 comes first.

A detail worth stating because it caused a real bug: when a stall and a flush both want the same pipeline slot, the flush must win. A stalled instruction on a path that turns out to be wrong must still be thrown away.

The choices, spelled out

- **Forward from two places, not one.** A value can be caught either one stage behind or two, so there are two forwarding paths. What is forwarded is the *final* result the instruction will write, not just the arithmetic output, so results from jumps and control registers forward correctly too.
- **Load-use: stall exactly one tick, then forward.** The only hazard that cannot be wired away, because the value does not exist yet.
- **A load feeding a store's data: stall, do not forward.** Forwarding it would need the value carried an extra stage plus another mux, to save one tick on a rare pattern. Deliberately left undone and written down, rather than built because it was possible.
- **When a stall and a flush want the same slot, the flush wins.** A stalled instruction on a path that turned out to be wrong must still be thrown away.

What it cost: Forwarding: free. A load feeding the next instruction: one tick, always. A taken branch: two instructions thrown away, every time.

Chapter 4: Guessing the future

At a fork in the road you have to pick a direction before you can see which one is right. That is the machine's position at every branch. It must keep fetching instructions, and it does not yet know which way the branch will go.

So it guesses. If the guess is right, nothing is lost. If it is wrong, everything fetched since is thrown away.

How it works

All three together are the **branch predictor**, and running ahead on what it says is **speculative execution**. When it is wrong, the wasted time is the **misprediction penalty**.

Three structures, because there are three different questions.

Where does this jump go? The **branch target buffer**, or **BTB**, is a table of addresses recording *last time we were here, we jumped to there*. It also records what *kind* of jump it was, which turns out to matter more than anything else in this chapter.

Will this branch be taken? A table of **two-bit saturating counters** that lean towards taken or not taken, and shift one step each time they are proved right or wrong. Two bits rather than one so a single surprise does not immediately flip a branch that is usually predictable: the counter has to be wrong twice. The clever part is that a branch is not looked up by its address alone. The address is mixed with a **global history register** recording how the last ten branches actually went, so the same branch gets different counters in different situations, and the machine can learn patterns like *this loop exits on the third time round*. Mixing the two with an exclusive-or is what gives the scheme its name, **gshare**.

Where does this return go? This one cannot be a table. A function called from three different places has to return to three different addresses, so a table indexed by the address of the return instruction has no single right answer to store. What works is a **stack**: a call pushes the address to come back to, and a return pops it. The structure is called the **return address stack**, or **RAS**.

bit in the whole predictor, and the reason is that a call and a plain jump look almost identical in the instruction encoding: they differ only in whether they save a return address. Without that bit, every plain jump pushed a return address that no return would ever remove, and the stack drifted permanently out of step from that moment on. On a small test with a plain jump inside the loop, return prediction was right 20 percent of the time without the bit and 97 percent with it.

The return stack, and what happens when it runs out

Eight entries is not many. Real programs nest deeper than that all the time, so what happens on the ninth call is a real decision, and the obvious answer is the wrong one.

Start with what the stack holds when it is full. Entry one is the *outermost* call, made first and returned to last. Entry eight is the *innermost*, the most recent call, and the first one that will return. The top of the stack is therefore always the call that will come back soonest.

The obvious answer, and why it is wrong. Let the pointer wrap, so the ninth call overwrites entry one. That destroys the single most valuable thing in the structure. Entry one is the outermost frame, the one the program will not return to until the very end, and therefore the one that must survive longest. Wrapping means deep calls eat shallow frames, which is exactly backwards. Worse, the damage is permanent: nothing ever restores that entry, so the stack is wrong from its first overflow until the program ends.

What this design does instead. On a call that does not fit, the stack is not touched at all. All eight entries stay exactly as they are. Instead a counter is incremented, recording that one call has gone unrecorded. A second call that does not fit makes it two, and so on.

That counter is the answer to an obvious question: how does the machine know which returns it has no entry for? It does not know *which*, only *how many*, and that is enough, because calls and returns nest. The calls that were dropped were the most recent ones, so the returns matching them are the first ones to come back. When the structure is a stack, a count is a complete record.

Coming back out. Each return that arrives while the counter is non-zero decrements it and leaves the pointer alone. So the counter is spent first, one return at a time, and only when it reaches zero does the pointer start moving down through the eight real entries. At that instant the pointer is exactly where it was before the overflow began, and the entries underneath were never disturbed. The stack heals itself completely.

So what does it predict for the returns it has no entry for?

It offers the top of the stack, over and over, once for each unit of overflow.

That sounds like a guess made out of nothing. It is not, and it is worth seeing why on a concrete example.

Suppose `main` calls a function that then calls itself eleven times, so twelve calls are outstanding and the stack holds eight.

0x1000	<code>main</code> calls the function
0x1004	so this is the address it returns to
0x2010	the function calls <i>itself</i>
0x2014	so this is the address it returns to

Every recursive call is *the same instruction*, so all eleven of them push the identical address 0x2014. Only the original call from `main` pushes something different.

Going down, the first eight calls fill the stack: entry one holds 0x1004, entries two through eight hold 0x2014. Calls nine to twelve do not fit, so the stack is untouched and the counter reaches four. The top of the stack is 0x2014.

Coming back out, the returns arrive innermost first:

return	counter	prediction offered	right?
1	4 → 3	top of stack, 0x2014	yes
2	3 → 2	top of stack, 0x2014	yes
3	2 → 1	top of stack, 0x2014	yes
4	1 → 0	top of stack, 0x2014	yes
5	0	entry eight, 0x2014	yes
6 to 11	0	entries seven down to two, 0x2014	yes
12	0	entry one, 0x1004	yes

Twelve out of twelve correct, on a stack that could only hold eight.

The four guesses were not lucky, they were guaranteed. The calls that did not fit all came from the same place in the program, so the address each of them *would* have stored is the address already sitting on top of the stack, put there by the last call that did fit. Repeating the top entry four times is not an approximation of the four missing entries. It is an exact reconstruction of them.

And notice return five, where the counter has just reached zero. The top of the stack is still the right answer, this time as a real stored entry rather than a repeat. The handover from guessing to remembering is seamless, because the pointer was frozen at precisely the place the guessing started.

Why it costs nothing to try. Even when this is wrong it loses nothing, because a wrong prediction and no prediction cost exactly the same here. Either way the machine works out the real return address a few cycles later and restarts fetching from there. Offering an answer can fail to win, but it cannot do worse than staying silent, so there is no reason not to offer one.

Where it does fail. Deep nesting through *different* functions: **a** calls **b** calls **c**, each from a different place, so every dropped entry held a different address and the top of the stack is wrong for all of them. Nothing but more storage fixes that. The saving grace is that the thing which actually overflows return stacks in practice is recursion, and recursion is precisely the case this handles perfectly.

Measured on a benchmark deliberately mixing shallow calls, thirty-deep recursion and twelve-deep nesting through distinct functions: counting with repeat-the-top predicted 75 percent of returns correctly, in 2,251 cycles. Letting the stack wrap managed 34 percent, in 2,447.

One limit is worth stating. The counter is six bits, so past a nesting depth of about seventy it stops counting up. On the way back out it then reaches zero too early, the pointer starts moving while returns are still unaccounted for, and the stack is out of step. Nothing in these workloads goes near that depth.

What was left out

Predicting indirect jumps. There are four kinds of control transfer and this design predicts three of them. A conditional branch needs a direction and a target, and has both. A direct jump or call needs a target, and it is written inside the instruction. A return needs a target that is nowhere in the instruction, and the stack supplies it, for the specific reason that a return goes back to wherever the call came from and calls nest. That is exactly the shape a stack models.

The fourth kind is every *other* indirect jump: a call through a function pointer, a virtual

method call, a compiled jump table. Their target sits in a register and is computed while the program runs. It can be different on every execution, and nothing was pushed anywhere, so no stack can help. Predicting these needs a dedicated structure holding several targets per call site and choosing between them from the path taken to reach it. That was left outside the scope of this design: it needs tagged, history-indexed storage with its own allocation and replacement policy, and it has to answer in the fetch stage within a single cycle, which is more machinery than the three structures above put together.

What happens instead is that an indirect jump falls back on the generic mechanism. The BTB hands back whatever that site jumped to last time, which is perfect for a call site whose target never moves, and wrong every single time for one whose target rotates.

Three refinements to the return stack, deferred. Each would help, each was measured against its cost, and each is written down rather than quietly skipped.

- **Counting repeats inside the stack.** Deep recursion fills all eight entries with the same address, which is wasteful. Counting repeats instead would stretch the effective depth considerably. It also adds per-entry state that must be saved and restored on every wrong guess.
- **Pushing earlier, at decode.** The first time a call site ever executes, the BTB does not yet know it is a call, so nothing is pushed and the matching return is mispredicted. Deciding at decode instead would close that, at the cost of splitting speculative state across two stages.
- **Restoring the stack's contents, not just its pointer.** Recovery after a wrong guess rewinds the pointer and the overflow counter, which is enough unless a wrong-path call actually overwrote a live entry. That case cannot be undone, and it is the residual few percent. Production cores of this class ship the same approximation.

What it cost: One bit that tells a call from a jump moved return accuracy from 20 percent to 97 percent. Keeping an overflowed stack intact and repeating its top entry beat letting it wrap, 75 percent against 34 percent.

Chapter 5: When things go wrong

Sometimes the machine has to stop what it is doing, deal with something else, and then carry on as though nothing had happened.

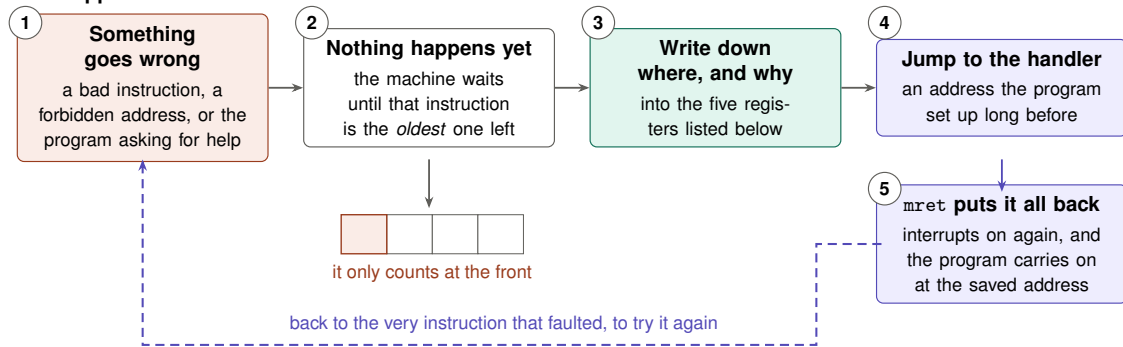
There are two reasons, and they have different names. An **exception** is the instruction's own fault: it is not a real instruction, or it asks for an address it is not allowed to touch. Exceptions are **synchronous**, meaning each belongs to one specific instruction and would happen again if the program were re-run. An **interrupt** comes from outside, like a timer going off, and is **asynchronous**: it has nothing to do with whatever instruction happened to be running. The word **trap** covers both.

Both end up in the same place, and the difficult part is the same in both cases: coming back afterwards as if nothing had happened.

How it works

The machine keeps a handful of special registers for this, called **control and status registers**, or **CSRs**. One holds the address to come back to, one a number saying what happened, one the

A trap is the machine stopping in the middle of a program, doing something else, and coming back as if nothing had happened.



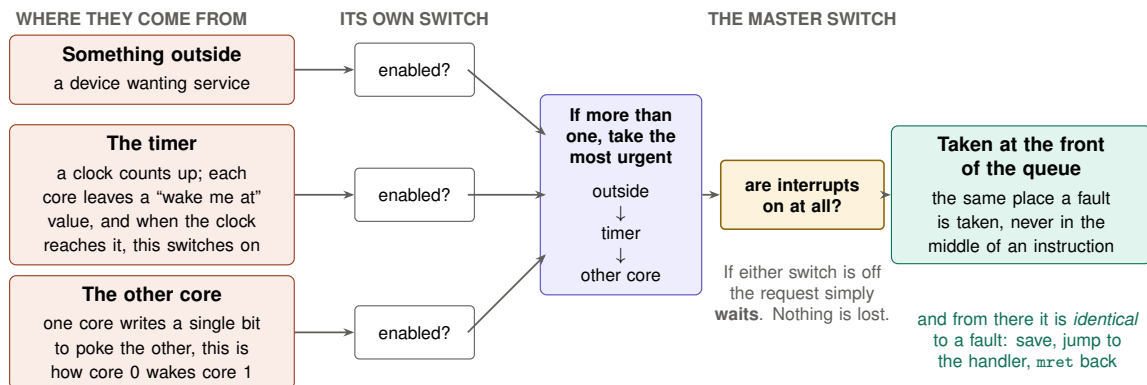
The five registers a trap uses, ordinary registers with reserved names.

`mepc` where we were, the address to come back to
`mcause` what happened, a number saying which kind
`mtval` the detail, the address or instruction involved
`mtvec` where the handler lives, set once at start-up
`mstatus` are interrupts on? A trap turns them off and remembers

Why wait for step 2? Because until the faulty instruction is the oldest, the machine cannot be sure it should have run at all, the branch that led to it might still turn out to be a wrong guess, or an older instruction might fault first. Acting only at the front of the queue is what makes the answer *exactly* right rather than nearly right.

Figure 8: **An instruction goes wrong.** The important step is the second one. The machine does not act on the problem when it is discovered. It waits until that instruction is the oldest one still in the machine, because until then, it cannot be sure the instruction should have run at all. Only then does it write down where it was and why it stopped, and jump to the handler. One instruction, `mret`, puts everything back.

An interrupt is the outside world asking for attention. Two switches decide whether the machine listens, and one rule decides *when*.



The timer is deliberately not cached. A cached clock would be read once, kept, and returned for ever after, a clock that stops, and a timer interrupt that never fires again. Being uncached is a property of the *device*, not of its address.

One bit tells the handler which kind it was. A fault and an interrupt land in the same five registers and run the same handler. The top bit of the "why did this happen" register is what separates them.

Figure 9: **Something outside asks for attention.** Interrupts come from a timer, from the other core, or from a device. Two separate switches have to be on before the machine will listen, and if either is off the request simply waits. It is never lost. Once it is allowed in, it is taken at exactly the same moment a faulting instruction is, and handled by exactly the same machinery.

address the handler lives at, and one whether interrupts are switched on. All of this happens in **machine mode**, the most privileged of RISC-V's privilege levels and the only one this design implements.

When something goes wrong, the machine fills those in, switches interrupts off, and jumps to the handler. The handler does whatever is needed. Then a single instruction restores the switch and jumps back.

from here on, how it was actually built

The word for getting this right is **precise exceptions**. It means that when the handler starts, the machine must look exactly as though every instruction before the faulting one finished completely, and no instruction after it had started at all.

That is easy in a simple machine and hard in this one, because instructions are finishing out of order all the time. The solution is the rule in figure 8: a fault is only acted on when the instruction reaches the front of the queue. At that moment everything older has genuinely finished and everything younger can be thrown away without ceremony.

Interrupts revealed a subtler problem. If an interrupt is allowed to land on an instruction that is part-way through writing to memory, that write can end up happening twice, once before the interrupt and once after the handler returns. For ordinary memory this is invisible. For a device it is corruption: a command sent twice.

A directed test that swept the arrival time of an interrupt across a loop of stores caught it: **five duplicated stores in twelve interrupts**. The fix is to hold interrupts back until the instruction retiring is not a memory operation. That delays an interrupt by a few cycles at most, and it cannot cause an interrupt to be starved for ever, because any loop of stores must eventually finish a branch.

Faults are deliberately *not* delayed the same way. A fault on a store means the store itself was wrong, and that is detected before anything is written.

The choices, spelled out

- **Control registers are read early and written at the very end.** The read happens in execute, so the old value can forward like any other result. The write happens only when the instruction actually retires. That split is what makes a trap precise: an instruction that is thrown away can never have changed a control register.
- **Interrupts are switched off on entry and remembered.** A trap pushes the previous enable state into a second bit, and the return instruction pops it back. That is the whole of the nesting mechanism.
- **Interrupts wait for an instruction that is not a memory access.** Otherwise a store can be executed twice, once before the interrupt and once after the handler returns. Invisible for ordinary memory; corruption for a device.
- **Faults are deliberately not delayed the same way.** A fault on a store means the store itself was wrong, and that is detected before anything is written.

What it cost: Five duplicated stores in twelve interrupts, before interrupts were held back until a non-memory instruction retires. The fix costs a few cycles of extra delay and cannot starve.

Chapter 6: Memory, and making it look fast

Memory is big and slow. Registers are tiny and instant. Almost everything interesting about a processor's performance is about hiding the difference.

The trick is that programs are predictable, a property called **locality**. If a program reads something it will very likely read it again shortly (**temporal locality**), and very likely read the thing next to it too (**spatial locality**). So keep a small fast copy of whatever was used recently, and most requests never travel far. The stack of ever-larger, ever-slower stores that results is the **memory hierarchy**.

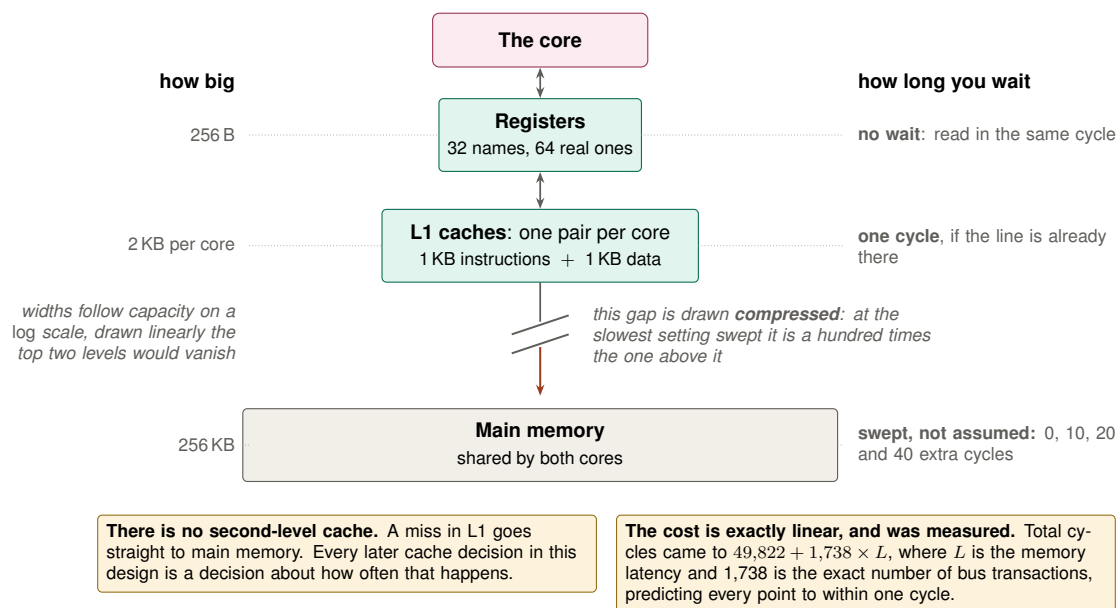


Figure 10: **Three levels, and a long way down.** Registers answer in the same tick. The caches answer in one tick, if what you want is there. Main memory is a different order of thing entirely, and this design has nothing in between, so a miss goes straight there. That is why the cost of a miss was swept across four settings rather than assumed.

How it works

A cache is a small table of recently used chunks of memory. Each chunk is a **cache line**, or block; here, sixteen bytes. When the processor asks for an address, the cache must answer two questions very quickly: do I have this (a **hit**) or not (a **miss**), and if so, where is it in my table?

Before either question can be answered there is a prior one to settle: when a line is brought in, *where in the cache is it allowed to sit?* The cache is thousands of times smaller than memory, so many different addresses have to share the same space.

There are two extreme answers, and both are bad.

Let a line sit anywhere. Then finding it later means checking every slot in the cache, every time. Thorough, and far too slow to do in a single cycle.

Or give each address exactly one permitted slot, worked out from the address itself. Now the check is trivial: look in that one place, and if it is not there it is not in the cache. But two frequently used addresses that happen to map to the same slot will now evict each other for ever, while the rest of the cache sits empty watching them fight.

The useful answer is in between: allow a line a small number of places. In this design, two. The address chooses a small group of slots, the line may sit in either slot of that group, and both are checked simultaneously. The group is called a **set**, the number of slots in each set is the number of **ways**, and a cache built this way is **set-associative**. This one is two-way set-associative. The two extremes have names as well: one slot per set is **direct-mapped**, and let-it-sit-anywhere is **fully associative**.

With that decided, the address splits into three parts, and what each part does follows from it. The **index** bits choose the set. The **tag** bits are stored next to the line itself, so the cache can check whether what is sitting in that set is really the line being asked for or some other address that maps to the same set. The lowest bits are the **offset**, choosing a byte within the line once it has been found.

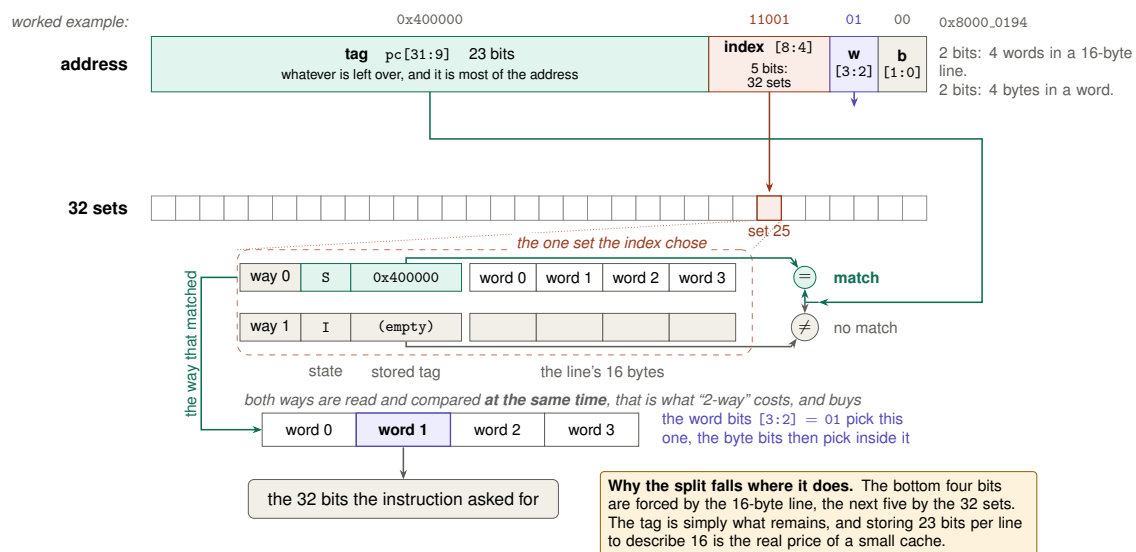


Figure 11: **How an address turns into an answer.** The address is split into pieces, and the size of each piece is forced by the shape of the cache: the bottom four bits because a chunk is 16 bytes, the next five because there are 32 rows. Both slots in the chosen row are checked at the same time, that is what having two slots costs, and buys. Everything left over is stored as a label. Storing 23 bits to describe 16 bytes is the real price of a small cache.

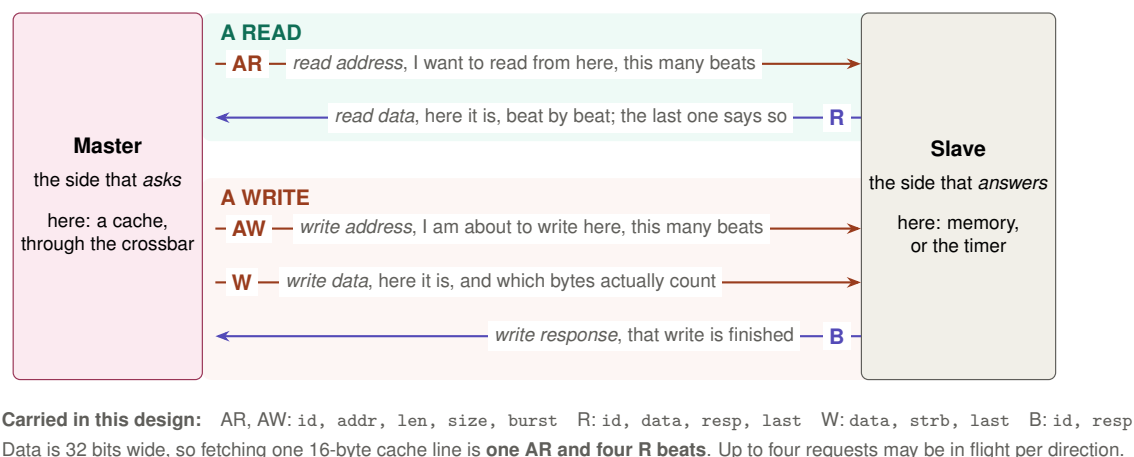
On a miss the cache has to go and fetch the line, and that means talking to memory over a **bus**. A bus is an agreed way for two pieces of hardware to pass messages without either knowing anything about the other. The one here is **AXI4**, a widely used standard. The side that asks is the **master**, the side that answers is the **slave**, and one line fetch becomes a **burst** of four transfers, each transfer being a **beat**.

from here on, how it was actually built

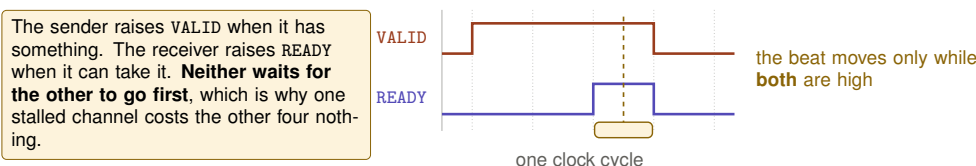
Each cache is 1 kilobyte, holding 16-byte chunks, with two slots per row, so 32 rows. That is small on purpose. The point of the project was to build the whole structure correctly, and a small cache exercises the interesting paths much more often than a large one.

There is no second-level cache. A miss goes straight to main memory. The bus carries a 16-byte chunk as four separate 32-bit transfers.

Memory latency is not a fixed number in this design, it is a setting, swept from instant to forty cycles, because a design that is only ever measured at one memory speed has only been half measured.



All five work the same way, and that way is the whole protocol:



Five independent tracks, not one bus taking turns. A read can overtake a write that is still finishing. Address and data travel separately, so the next address can go out while the current data is still arriving. And a write needs its own channel B to be told it worked, precisely because, unlike a read, it has no data coming back to carry the news.

Figure 12: **The bus, and the one rule that runs it.** Five separate channels, not one shared wire. Because they are separate, a read can overtake a write that is still finishing, and the next address can go out while the current data is still arriving. The handshake at the bottom is the whole protocol: the sender says it has something, the receiver says it can take it, and the transfer happens on the tick when both are true. Neither side ever waits for the other to go first.

What came out of that sweep is the cleanest result in the project. Total running time turned out to be exactly

$$49,822 + 1,738 \times L$$

cycles, where L is the memory latency. The 1,738 is not fitted, it is the exact count of trips the machine made to memory. That formula predicted every measured point to within a single cycle.

This is the kind of result worth more than a raw performance number, because it says exactly where the time goes: not to some vague inefficiency, but to a countable number of journeys, each costing exactly the latency.

The choices, spelled out

- **Write-back, not write-through.** This is the first question anyone asks about a cache, and the two answers are genuinely different machines. A *write-through* cache passes every store straight on to memory as well as keeping it locally: memory is always correct, nothing ever has to be remembered, and every single store costs a trip to memory. A *write-back* cache keeps the change to itself and marks the line *dirty*; memory is only brought up to date when that line is evicted, or when the other core asks for it. Write-back costs one extra bit per line, a path for writing lines out, and the obligation to remember that memory is stale. It buys back a trip to memory on every repeat store to the same line, which is most stores. This design is write-back, and chapter 8 depends on that: “I have

changed this and nobody else knows” is exactly the Modified state.

- **Write-allocate.** When a store misses, the line is fetched into the cache first and then written. The alternative is to send that one store to memory and leave the cache alone. Write-allocate pays for a fetch nobody asked for, and wins whenever the program writes near the same place again shortly, which for stacks and arrays is almost always.
- **Blocking.** One miss at a time. While a miss is being serviced the cache refuses new requests. A non-blocking cache keeps serving hits underneath a miss, which needs a set of registers tracking outstanding misses and is where a great deal of complexity lives. This one does the simple thing, and the out-of-order core above it hides part of the cost anyway by finding other work.
- **Two ways, with one bit of history per set.** Each address can live in either of two slots. When both are full, the one used less recently is replaced. At two ways that policy is a single bit, which is the size where being exactly right is cheaper than being approximately right. The bit is updated on every hit, not just on fills, and a free slot is always used before the bit is consulted.
- **Tags in flip-flops, data in memory arrays.** The tag store here is small enough that holding it in flip-flops is free, and that buys three things: the hit decision is available in the same cycle, the whole cache can be invalidated in one tick when a program modifies its own instructions, and reset genuinely empties it. Production caches with far larger tag stores cannot afford this. The coupling is worth recording, because it is invisible in the code: move the tags into memory arrays and the one-cycle invalidate quietly becomes a walk over every entry.
- **Eviction is serialised.** If a dirty line has to be thrown out to make room, it is written back *completely* before the replacement is requested. One bus transaction in flight at a time, which matches the single adapter underneath.
- **Devices are uncached because of what they are, not where they are.** Accesses to the timer and other devices bypass the cache entirely. Deciding that from a single address bit would be simpler and wrong: the timer sits at a low address, and a cached clock is read once and then frozen forever, which stops time and kills every future timer interrupt.
- **A deliberately reduced bus.** The five channels are implemented in full, but the optional signals for locking, cacheability hints, protection and quality-of-service are left out. They are all optional, none of them changes what this system does, and each one would be a channel of untested logic.

What it cost: Every extra cycle of memory latency cost this program exactly 1,738 cycles, one per trip to memory. Across the sweep, performance fell from 0.736 to 0.510 instructions per tick.

Chapter 7: Letting instructions go out of order

Here is the waste that is left. In the pipeline of chapter 2, instructions go through in the order they were written. So if one of them is stuck waiting for memory, everything behind it is stuck too, including instructions that need nothing at all from it and could have run twenty cycles ago.

The fix sounds reckless and is not. Let instructions run as soon as their inputs are ready, in whatever order that turns out to be, then put the results back in the right order at the end,

so that from outside nothing looks unusual. This is **out-of-order execution** with **in-order commit**. It is also called *dynamic scheduling*, because the hardware decides the order while the program runs, rather than the compiler deciding it in advance.

Separately, this machine is **two-wide superscalar**. Superscalar means starting more than one instruction in the same clock cycle, and two-wide means two of them. The two ideas are independent: a machine can be superscalar and still in order, or out of order and only one wide. Combining them is what makes the structures in this chapter necessary, and it is worth keeping the two sources of difficulty apart, because they cause different problems.

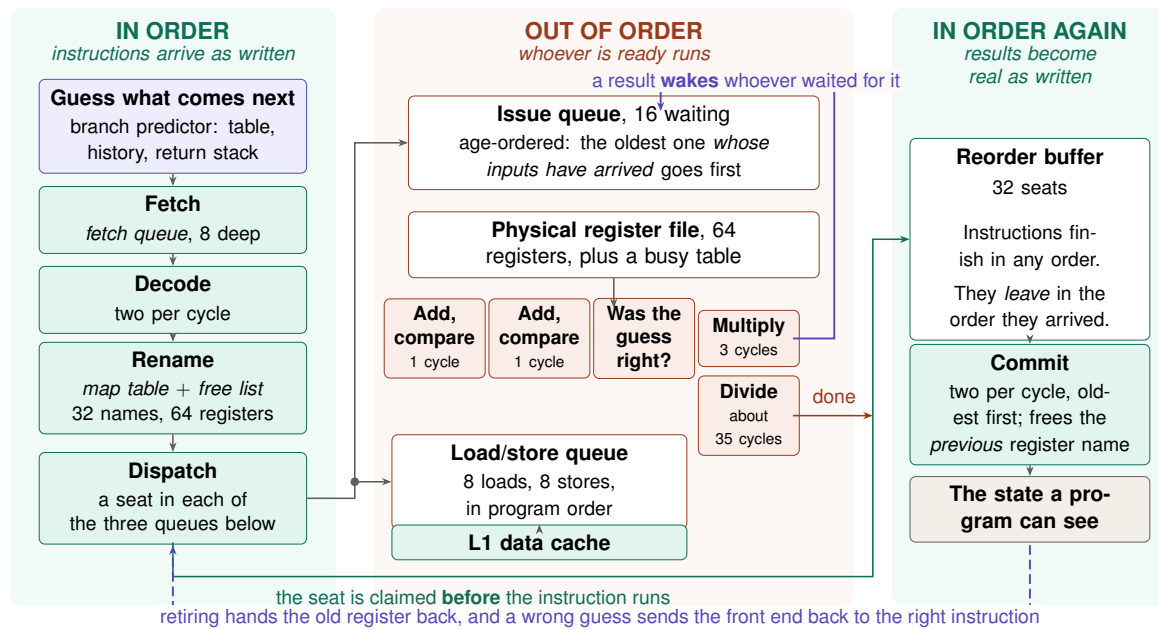


Figure 13: **The whole machine, in three parts.** Instructions arrive in order on the left. In the middle they run in whatever order their inputs happen to arrive, which is the entire point, and why a slow load no longer blocks the fast work behind it. On the right they are put back into order before anything becomes permanent. The purple loop is the mechanism that makes it work: a finished result immediately wakes everyone who was waiting for it.

Three kinds of dependency, and why only one of them is real

If instructions may run in any order, the machine has to know which orderings would change the answer. There are exactly three ways two instructions can clash over a register, and telling them apart is the single most important idea in this chapter.

Take three instructions:

	instruction	
(1)	add x1, x2, x3	writes x1
(2)	sub x4, x1, x5	reads x1
(3)	add x1, x6, x7	writes x1 again

Read after write. Instruction (2) reads x1, which instruction (1) produces. Until (1) has computed its result, (2) simply has nothing to work with. This is a **true dependency**, and it is the real one. A value has to physically travel from one instruction to the other, and no amount

of hardware cleverness can make it arrive before it exists. Every machine in this document, from chapter 1 onwards, has to respect it.

Write after read. Instruction (3) writes `x1`, which instruction (2) still has to read. If (3) runs early and overwrites `x1`, then (2) reads the wrong value. This looks like a dependency and is not. Nothing flows from (2) to (3). Instruction (3) does not want anything (2) has; it merely wants to use the same *name*.

Write after write. Instructions (1) and (3) both write `x1`. When everything settles, `x1` must hold (3)’s value, because (3) came later. If they complete out of order, it will hold (1)’s. Again, nothing flows between them. They are fighting over a name.

That is the whole insight. The second and third are called **false dependencies**, and the word false is exact: they are artefacts of the register file being small. RISC-V gives a program 32 register names, and a compiler reuses them constantly, not because instruction (3) has anything to do with instruction (1) but because there were no other names available.

So: if the shortage of names is what creates the conflict, the cure is more names.

from here on, how it was actually built

Renaming: buying more names

Underneath the 32 names the program can see, this machine has 64 real registers. Every instruction that produces a result is handed a fresh one, and a **map table** of 32 entries records which real register each name currently stands for.

Rename the three instructions above and the problem evaporates:

	as written	after renaming	
(1)	<code>add x1, x2, x3</code>	<code>add p33, p7, p9</code>	
(2)	<code>sub x4, x1, x5</code>	<code>sub p34, p33, p12</code>	still waits for (1)
(3)	<code>add x1, x6, x7</code>	<code>add p35, p18, p21</code>	now independent

Instruction (3) writes `p35`, which nothing else touches, so it can run whenever it likes: immediately, before (1), before (2), it no longer matters. The write-after-read and write-after-write conflicts have not been *solved*, they have ceased to exist. What survives is exactly one edge, (1) to (2), and that is the real one.

Two supporting pieces make this work. A **free list** holds the real registers nobody is currently using, and rename pops one for each result. And when an instruction finally retires, it hands a register back.

Which register it hands back is worth stating carefully, because getting it backwards is the classic first-out-of-order-processor bug. It does not free the register it just wrote: that one holds the current value of `x1` and is very much in use. It frees the one that `x1` pointed at *beforehand*. That older register is only now provably dead, because this instruction has retired, so every instruction that could still have wanted the old value has already finished. Freeing the new one instead throws away the result the instruction just computed.

And now the superscalar wrinkle. Two instructions are renamed in the same cycle, and they may well depend on each other. If instruction (2) sits in the same pair as instruction (1), then (2) must be given (1)’s brand-new register as its source, even though the map table has not been updated yet at the moment (2) reads it. The same applies to the write-after-write case: (3) must record (1)’s register as its previous mapping, not whatever the table happens to say, or commit frees the wrong one and a live register silently leaks. Both are resolved within the cycle,

by folding each slot's lookups over the remaps of the older slots beside it. This is a difficulty created purely by issuing two at a time; at one instruction per cycle it does not exist.

The structures, one at a time

Here is everything an instruction meets, in order, and the decision behind each.

- **The fetch queue**, 8 deep. It separates *asking* for instructions from *using* them, so the front end can keep requesting while the rest of the machine is busy. One request is outstanding at a time.
- **Decode**, two instructions per cycle, which also builds the constant values carried inside instructions.
- **Rename**, as above: a 32-entry map table and a free list, 32 names onto 64 registers. Register zero is pinned so it can never be renamed, even by a mistake elsewhere.
- **The reorder buffer**, 32 seats, arranged as a ring. Two seats are claimed at the back each cycle, instructions mark their own seat done in whatever order they finish, and up to two leave from the front when they are the oldest. This is the structure that makes the machine look ordinary from outside, and it is where *precise state* comes from: at the moment an instruction retires, everything before it has genuinely happened and nothing after it has.
- **The issue queue**, 16 entries. When any result appears anywhere in the machine, the number of the register it wrote is broadcast, and every waiting instruction compares that number against the registers it is waiting for. This is a small associative search, done every cycle, and it is what *wakeup* means.

Two decisions here. First, readiness is **edge-driven**: an entry becomes ready when a matching broadcast arrives, rather than by repeatedly asking whether its inputs are available. That is much cheaper, and it creates one awkward case that has to be handled explicitly, because an instruction can arrive in the queue in the very cycle its input is being broadcast, and would otherwise wait for a wakeup that has already gone past. Second, among the entries that are ready, the **oldest wins**, and the queue physically shifts to close its gaps so that position order is age order. Choosing by position instead would be cheaper still, and would let a young instruction starve an old one indefinitely.

- **The physical register file**, 64 registers, with a *busy table* recording which results have not arrived yet. Four read ports, since two instructions each read two sources, and three write ports.
- **The execution units**: two general arithmetic units, a branch unit that checks whether the guess was right, a multiplier taking three cycles, and a divider taking about 35.
- **The load/store queue**, 8 loads and 8 stores, both kept as rings in program order. This is where memory ordering is enforced, and it gets its own section below.
- **Four branch checkpoints**, for the recovery in figure 14.

Multiply and divide

The base RISC-V instruction set has no multiply and no divide. That is deliberate: a minimal processor can leave them out and let software synthesise them from shifts and adds, slowly.

Adding them is a standard extension, and it is the **M** in RV32IM. It brings eight instructions: a multiply in four flavours, and divide and remainder in signed and unsigned forms.

Why four flavours of multiply. Two 32-bit numbers multiplied together produce a 64-bit result, and the machine has nowhere to put 64 bits. So the instruction set splits the job: one instruction returns the low half, and three return the high half, differing in whether each operand is treated as signed or unsigned. Programs that just want ordinary arithmetic use the low half and never think about it. Programs doing wide arithmetic ask for both halves and stitch them together.

The natural way to build that is four datapaths, and it is the wrong way, because signedness is exactly the kind of detail that is easy to get subtly wrong in one of four places and hard to notice. This design uses a *single* 33-bit signed multiplier for all four. The extra bit above 32 carries the sign, and each flavour is expressed by choosing what to put in that bit for each operand, and then choosing which half of the product to keep. A per-flavour signedness bug is not merely unlikely, it is structurally impossible: there are not four paths to disagree with each other, there is one path and two configuration bits.

It takes three cycles, and it is a free-running pipe with no handshake and no valid signal. The result simply arrives three stages later, and correctness comes from the pipe being locked in step with the rest of the machine rather than from control logic tracking it. This is also the origin of a detail from earlier in this chapter: because a multiply result surfaces three cycles late, it can land in the same cycle as a fresh single-cycle result, which is why the register file has a third write port.

The divider is the opposite kind of structure. It is a restoring divider that produces *one bit of quotient per cycle*, so a 32-bit divide takes 32 iterations, and about 35 cycles all told. It is not pipelined, so a second divide simply waits.

Signed division is handled by taking both operands to absolute values, dividing as unsigned, and fixing the signs at the end: the quotient is negated when the operand signs differ, and the remainder takes the sign of the *dividend*, which is what the specification requires.

Two corners of division are worth calling out, because they are where specifications usually bite.

Dividing by zero does not trap. RISC-V defines exact results instead: the quotient is all ones, and the remainder is the original dividend. This design detects it when the operands are loaded and skips the loop entirely, so a division by zero is fast rather than slow. The subtlety is that the remainder must be the *original* dividend, not its absolute value, and that the sign-fixing step must be suppressed, or it would helpfully “correct” the all-ones quotient into a positive one.

The overflow corner is handled by not handling it. Dividing the most negative number by minus one has a mathematical answer that does not fit in 32 bits, and the specification says to return the most negative number again. This design has no special case for it, and that is a decision rather than an oversight: taking absolute values first makes the right answer fall out naturally, because the intermediate value wraps back to exactly the required result. Designs whose datapath does not take absolute values need an explicit flag for this case. The corner is tested directly rather than assumed.

Why a slow divider was the right choice here. A radix-2 divider is the simplest and smallest option, and 35 cycles is a long time. Faster dividers exist, produce several quotient bits per cycle, and are considerably larger.

The argument for keeping the slow one is the whole subject of this chapter: *under out-of-order execution, a long but fixed latency is a scheduling problem rather than a speed problem*. The divide does not block anything except instructions that actually want its result. Everything else slides underneath it. A machine that executes in order has no such escape, and there the same

divider would be straightforwardly expensive.

That claim was recorded together with the measurement that would disprove it, which is the useful part. In the in-order machine, a divide alone took 35 cycles, and a divide followed by ten independent additions took 45. Those ten additions cost ten cycles because nothing could start until the divide finished. In the out-of-order machine they are exactly the work that fills the gap.

What the divider taught the control logic. It was the first unit in the design that takes long enough to be interrupted part-way through, and it exposed a question the pipeline had never had to answer: what should happen if a trap arrives while a divide is still running? The answer adopted is that older instructions drain, the divider is cleared by the same signal that clears everything else, and the address recorded for the handler to return to is the divide's own, so it is simply executed again.

Getting that wrong is invisible to every test that does not interrupt a long-running operation, which is precisely why it was tested on purpose instead of being left to the general trap tests to stumble over.

Memory, which is the hard part

Registers are easy by comparison: an instruction names them explicitly, so the hardware can see the dependency immediately. Memory addresses are computed, so until a load or store has calculated its address, nobody knows what it touches. Working out which memory operations genuinely conflict is called **memory disambiguation**.

The safe answer is: a load may not resolve until every store older than it has at least worked out its address. Once they have, the load compares itself against each of them and takes one of three paths.

- **Same address, same size:** the value is taken straight out of that store, without going to memory at all. This is **store-to-load forwarding**, and it is common, because programs write a variable and read it back constantly.
- **Overlapping but not identical:** the load waits until that store has actually written to memory, then reads normally. Assembling a value from a partially overlapping store is possible and was not built.
- **No overlap:** the load goes to memory, and the store is irrelevant.

This policy is **conservative**, deliberately. The aggressive alternative lets a load go before older store addresses are known, and detects afterwards that it read something a store was about to change, then re-executes it. That is faster in the common case, where the load and the store were unrelated all along, and it needs a whole additional detection-and-recovery mechanism. It is written down as future work rather than half-built.

Two smaller decisions worth naming. Loads are processed in program order among themselves, because there is one memory port and nothing to be gained by reordering loads against each other. And committed stores drain to memory whenever the port is free, with loads given priority, because a committed store is architectural state that must eventually land, while a waiting load holds no resource and therefore cannot deadlock anything.

Structural hazards, and where they came back

Chapter 3 listed three kinds of hazard and said the structural one, two instructions wanting the same hardware at once, was the kind the five-stage pipeline simply did not have. That was true

of that machine and is not true of this one. Two instructions per cycle, arriving in any order, contend for everything, and this design answers that in three different ways.

- **Sometimes by building more hardware.** The register file has *three* write ports rather than two. The third exists for one specific collision: the multiplier takes three cycles, so its result can surface in the same cycle as a fresh single-cycle result from an arithmetic unit. Two ports would have meant one of them waiting. This is the expensive answer, and it was used once.
- **Sometimes by restricting what may go where.** The two issue ports are deliberately not equal. Port 0 will issue anything. Port 1 takes *only single-cycle integer arithmetic*: no memory, no branches, no multiply or divide, no control-register access. So two loads can never issue together, and neither can a load and a branch. There is a matching restriction at the other end of the machine: a second instruction is only dispatched alongside the first when *both* are simple arithmetic, which keeps the memory queue, the branch checkpoints and the control-register path entirely single-issue. Widening those is a measured follow-on rather than an oversight, and each has a specific missing piece named.
- **Sometimes by simply making everyone wait.** The divider is a single unit and is not pipelined. An instruction that needs it cannot even *ask* to issue while it is busy, so two divides are perfectly serial, about seventy cycles for the pair. The same answer appears again in chapter 8: one cache miss at a time, one coherence conversation at a time, and both cores sharing one route to memory.

The honest summary is that out-of-order execution does not remove structural hazards. It removes the requirement that they be resolved *in program order*, which is a different and much more useful thing: an instruction blocked on the divider no longer blocks the twenty behind it that want nothing from it.

Undoing a wrong guess

All of the above runs on speculation. The moment a branch is predicted, every instruction behind it is provisional, and when a prediction turns out wrong they all have to disappear as though they never ran.

Throwing away the results is the easy half: the discarded instructions simply never mark their seats done, and their seats are released. The hard half is the map table, because every one of those instructions renamed something.

The first method always works: step backwards through the reorder buffer, youngest first, and undo each instruction by restoring the register its destination pointed at before. Nothing extra needs to be stored, because that previous mapping is already recorded for the commit case. It costs one cycle per discarded instruction, and with a window 32 deep that measured 31.3 cycles.

The second method photographs the whole map table at every branch and restores it in a single cycle. That measured 3.85 cycles, an improvement of 8.1 times, and a deeply nested branch benchmark got 23.7 percent shorter.

There are four photographs, which is fewer than the window can hold branches. That is intentional. It guarantees the machine runs out regularly and falls back on stepping backwards, so the slow path is exercised constantly during ordinary work rather than sitting untested until the day it is needed.

Recovery is also not only about branches. A trap, an interrupt, a **fence**, a return from a handler and a failed atomic all need the same machinery, and they are ranked: an interrupt

The machine has guessed, and run ahead. Now the guess turns out to be wrong.



Throwing away the results is the easy part. The hard part is the **register map**: every one of those instructions renamed something, and the map has to go back to exactly what it was at the branch.

Method 1, walk it back

Step backwards through the window, youngest first. Each instruction already carries the name its destination held *before* it, so undoing it is just putting that name back.

Costs **one cycle per instruction**.

Method 2, keep a photograph

At every branch, photograph the whole map before running on. If the guess was wrong, put the photograph back.

Costs **one cycle**, however deep the window is.



What it cost, measured:

walking		31.3 cycles to recover
photographs		3.85 cycles, 8.1 times better. A deeply nested branch test shrank by 23.7 percent.

There are only four photographs, and that is on purpose. The window holds far more than four branches, so sooner or later a fifth arrives and the machine has to fall back to walking. Building it that way means the slow path runs constantly during ordinary work, rather than sitting untested until the day it is needed.

Figure 14: **Undoing a wrong guess, the hard half.** Throwing away the results is easy. Putting the register map back is not, because every discarded instruction changed it. Stepping back one instruction at a time always works and cost 31.3 cycles. Photographing the map at each branch costs one cycle however deep the window is. There are deliberately only four photographs, so the slow path stays exercised during ordinary running rather than sitting untested.

outranks a fault at the head, which outranks an instruction that needs the machine drained around it, which outranks a branch that guessed wrong. The ranking exists because the oldest instruction always wins, and a fault at the front of the queue is by definition older than any branch still in flight.

What it cost: Recovery from a wrong guess fell from 31.3 cycles to 3.85, 8.1 times better, and a deeply nested branch test got 23.7 percent shorter. Through the real cache hierarchy the two-wide machine reached 0.736 instructions per tick.

Chapter 8: Two cores that agree

Two cores should be twice the machine. They are not, and the reason is interesting.

The problem is that each core keeps its own private copy of the memory it is using. That is the whole point of a cache. But if both cores hold a copy of the same line and one of them changes its copy, the other is now holding something quietly wrong. Keeping them in agreement is **cache coherence**.

This is the worst kind of bug, because nothing looks broken. No error is reported. The program just gets an answer that was true a moment ago.

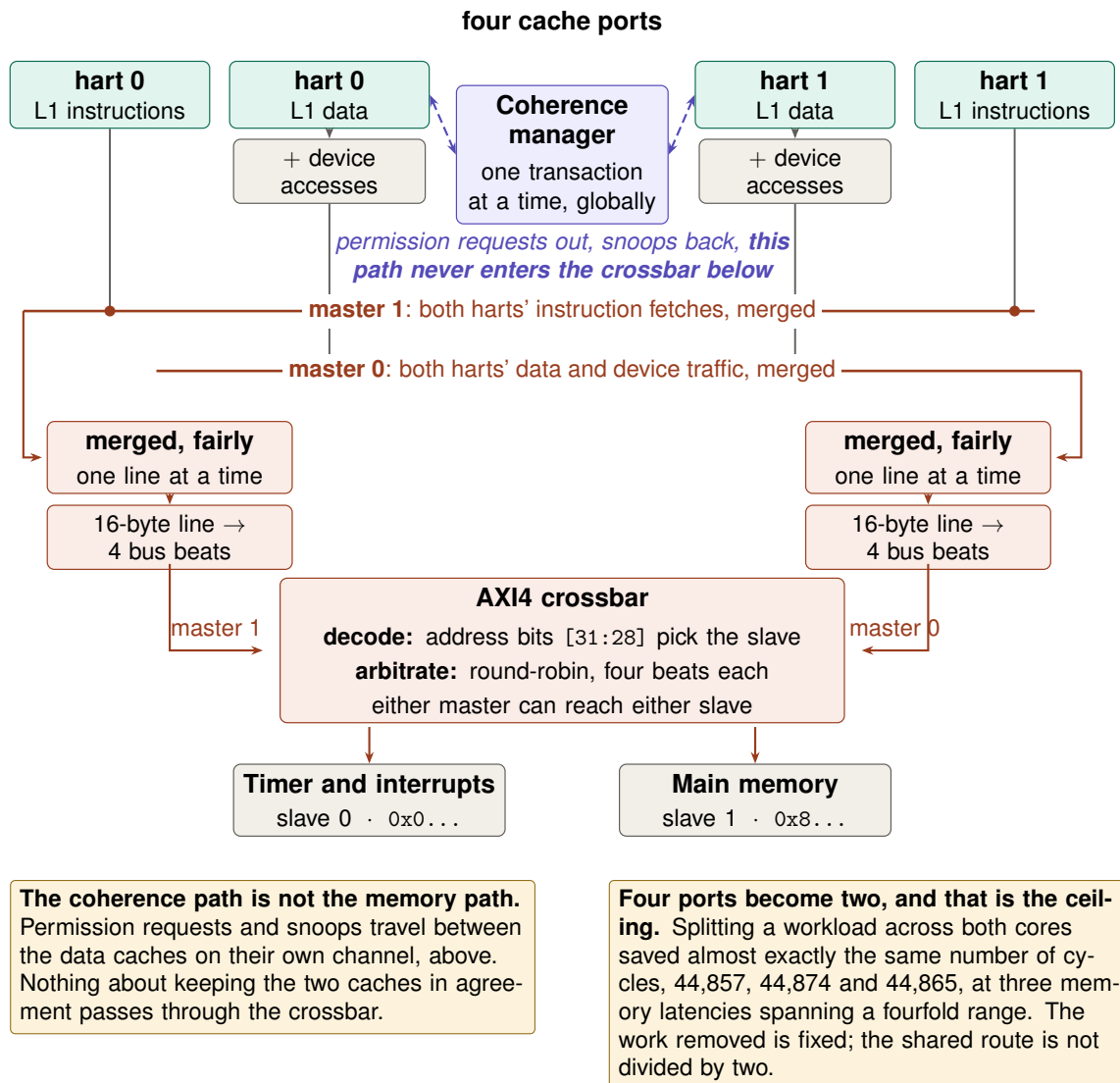


Figure 15: **How the two cores reach memory, and how they reach each other.** Two things are happening here and they are separate. The traffic going down through the crossbar is data moving between caches and memory. The purple path across the top is the caches asking each other's permission. It never touches the crossbar. Note also what happens on the way down: four cache ports become two. That funnel, not the coherence protocol, is what limits how much the second core helps.

How it works

Each cached line carries a note about how the cache is allowed to treat it, and there are four possibilities: I do not have it; I have it and so might someone else; I have it and nobody else does; I have it, nobody else does, and I have changed it. Those are **Invalid**, **Shared**, **Exclusive** and **Modified**, and their initials give the protocol its name: **MESI**.

They are kept honest by a simple discipline. Before a core may write, it must be sure nobody

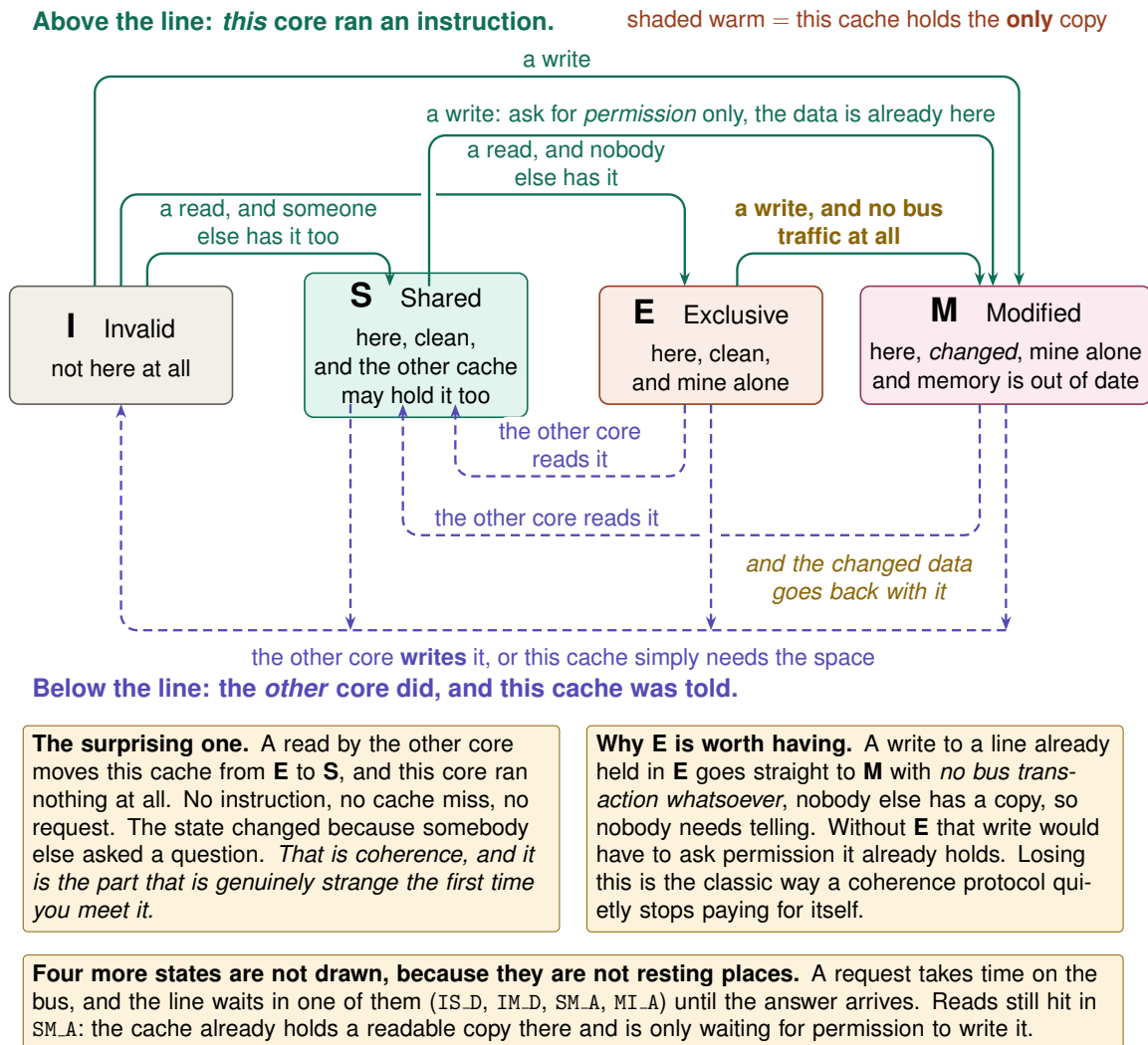


Figure 16: **The rules each cached copy follows.** Everything above the line happens because this core ran an instruction. Everything below happens because the *other* core did. The one to look at is a read by the other core moving this cache from “mine alone” to “shared”, this core ran nothing, and its state changed anyway. That is coherence in one arrow.

else holds a copy, so it asks, and any cache that has one gives it up. Before a core may read a line another core has modified, that core must hand the modified version over.

The asking is done by **snooping**: every cache watches the shared coherence channel and answers requests about lines it holds. The alternative is a **directory**, a central record of who has what, which scales further and costs more. The property all of this exists to guarantee has a name too: **single-writer, multiple-reader**. At any moment a line may have one writer and no readers, or many readers and no writer, and never both.

The surprising part, and the part that makes it coherence rather than just caching, is that a cache can be forced to change state without its own core doing anything at all.

— from here on, how it was actually built —

There is one point in the machine where these conversations are ordered, and it handles exactly one at a time. That is stricter than strictly necessary, two requests for unrelated addresses

could safely proceed together, and it was chosen deliberately. With one conversation at a time, a whole family of races becomes impossible by construction rather than by argument. With two blocking caches, the worst case is one extra transaction of delay.

The measurement is the interesting part. Splitting a workload across both cores gave a speedup of 1.583 with instant memory, but only 1.216 once memory was 20 cycles away. The obvious reading is that coherence gets more expensive as memory gets slower.

It is the wrong reading, and the number that shows it is in a column that is easy not to look at. The *absolute* saving was 44,857 cycles, then 44,874, then 44,865, essentially unchanged across a fourfold change in memory speed. The work the second core takes off the first is a fixed amount of computation, and it stays fixed. What grows is the total, because both versions pay the extra memory latency, and latency spent on a route that is shared cannot be divided by two.

So the honest statement is not *coherence costs more when memory is slow*. It is: **this design's parallel headroom is limited by the shared route to memory, not by the coherence protocol**. That is a statement about what to build next, which a speedup figure alone would not have given.

Separately, the cost of making the data cache coherent at all was measured directly, by running the same workload before and after: 56 cycles out of roughly 75,000. That is 0.07 percent.

What a second core adds to the hazard list

Chapter 3 named three kinds of hazard, and chapter 7 showed the structural kind coming back inside a single core once it went wide. A second core adds more, and also changes what the word can mean.

Structural hazards, everywhere below the caches. Look again at figure 15: below the private caches, every single thing is shared. Both cores' data traffic funnels into one bus master. Both cores' instruction fetches funnel into another. The crossbar carries two masters and two slaves. The coherence manager handles one conversation at a time, across the whole machine. And there is one memory. Each of those is two requesters and one resource, which is the definition, and together they are the ceiling this chapter measures.

Not data hazards, but races. Inside one core the hardware finds every dependency and honours it silently. Across two cores it does not, and cannot. What it offers instead is three separate guarantees that are easy to confuse:

- **Coherence** answers *do the two caches agree about the value at this one address*. That is MESI, and it is what this chapter is about.
- **Consistency** answers *in what order does the other core see my writes to **different** addresses*. That is a separate question with a separate answer, called a memory model, and getting it wrong produces programs that work until they do not.
- **Atomicity** answers *can I read a value, change it, and write it back without anyone slipping in between*. That needs its own instructions, which is what the reservation mechanism in the figures is for.

Coherence buys you neither of the other two. A design can keep every cache perfectly in agreement and still let one core observe another's writes in an order the specification forbids.

Two hazards that simply did not exist with one core.

The first is a new stall. With one core, a store to a line already in the cache was a hit and cost nothing. With two, a line may be held in the shared state, and a store to it must now stop

and ask permission before it can proceed. Loads to that line still hit while it waits; stores do not.

The second is **false sharing**, and it is the one that surprises people. Coherence works on whole lines, never on individual bytes. Two cores writing to two *different* variables that happen to sit in the same sixteen-byte line will bounce that line back and forth between them indefinitely. The two programs share no data and have no dependency whatsoever, and they will still slow each other down badly.

How the ordering question is actually checked. Not by inspection. There is a standard set of small two-core programs, called litmus tests, whose every permitted outcome is enumerated by a formal model of the RISC-V memory ordering rules. Each is run repeatedly across a sweep of memory timings, the set of outcomes actually observed is accumulated, and that set is compared against the model's. The comparison is deliberately one-sided: producing an outcome the model forbids is a hard failure, while never producing one it permits is merely reported. The second direction is not a bug, because a machine is always allowed to be more orderly than the specification requires.

The choices, spelled out

- **Four states, not five.** The protocol is MESI: Invalid, Shared, Exclusive, Modified. A fifth state, Owned, is defined in the code and never reached. Owned lets a dirty line be shared, with one cache responsible for eventually writing it back; it saves memory traffic and adds a responsibility that has to be tracked. It was left unreachable rather than deleted, and the code says so, because a state that exists and is never entered is a very different thing from a state somebody forgot.
- **One coherence conversation at a time, across the whole machine.** Strictly stronger than necessary: two requests for unrelated addresses could safely overlap. Allowing that would mean two snoop broadcasts and two sets of replies interleaving, which is the exact family of races that makes other designs need extra intermediate states. With one at a time those states are impossible by construction rather than by argument, and the cost with two blocking caches is at most one transaction of delay.
- **Asking permission travels on its own channel.** Snoops and their replies do not go through the crossbar. The data path and the coherence path are separate, which is the single most-missed thing about figure 15.
- **A write to a line you already own alone is silent.** Exclusive to Modified involves no bus traffic whatsoever, because nobody else has a copy and so nobody needs telling. Making that transition ask permission is the classic way a coherence protocol stops paying for itself.
- **A clean handover sends no data; a dirty one must.** If the other core wants to read a line this cache holds unmodified, memory already has the right contents, so only a note is sent. If this cache has changed it, the data goes with the reply, because otherwise the other core reads stale memory.
- **Throwing out a modified line is an ordered, acknowledged transaction.** Not fire-and-forget. The line is not considered gone until the writeback has been accepted, which is why there is a waiting state for it.

What it cost: 1.583 times faster on two cores with instant memory. But the saving was 44,857, 44,874 and 44,865 cycles at three memory speeds, essentially constant. Coherence itself cost 0.07 percent.

Chapter 9: Knowing that it works

Building it is half the job. The other half is being able to say, with evidence, that it is right.

You cannot test everything. A 32-bit processor has more possible states than there are atoms available to count them with. So the question is not *did I test it all*, it is *did I test the things that matter, and how would I know if I had not*.

What UVM is

UVM stands for the Universal Verification Methodology. It is not a tool and not a language. It is a standard library, plus a set of conventions, for building the thing that tests a design.

The problem it solves is that home-made testbenches all rot the same way. They start as a short program that pokes some wires and prints results. Then the design grows, the checks get tangled into the poking, the stimulus gets tangled into the checks, and eventually nobody can add a test without breaking two others. UVM's answer is to cut the testbench into pieces with fixed responsibilities, so that each piece can be written, replaced and reasoned about on its own.

The vocabulary is small, and worth learning because every arrow in the figure below is one of these:

- An **interface** is the bundle of wires between the testbench and the design. Nothing else touches the design directly.
- A **driver** takes an abstract request and wiggles the wires to carry it out. It knows the protocol; it does not know what to send.
- A **monitor** watches the same wires and turns what it sees back into abstract observations. It never drives anything.
- A **sequencer** decides what to ask for next, and a **sequence** is a written plan of requests. This is where *what to test* lives, kept apart from *how to test it*.
- An **agent** is a driver, monitor and sequencer bundled for one interface. An agent can be *active* (it drives) or *passive* (it only watches).
- A **scoreboard** decides whether what happened was correct. It receives observations from monitors and compares them against a reference.
- A **coverage collector** records which interesting situations actually occurred.
- The **environment** holds all of the above and wires them together. The **test** picks the environment's settings and starts a sequence.

The payoff is the separation. Because the monitors only observe, the same checking works no matter what stimulus is running, hand-written programs, compiled C, or randomly generated instructions. Because the checks live in scoreboards rather than inside the stimulus, adding a test cannot silently weaken the checking. And because everything is connected in one place, a component that was built but never wired up is visible as a missing line rather than as a quiet absence of failures.

That last point is not theoretical. A monitor whose output goes nowhere still runs, still samples, and still prints confident-looking statistics of its own.

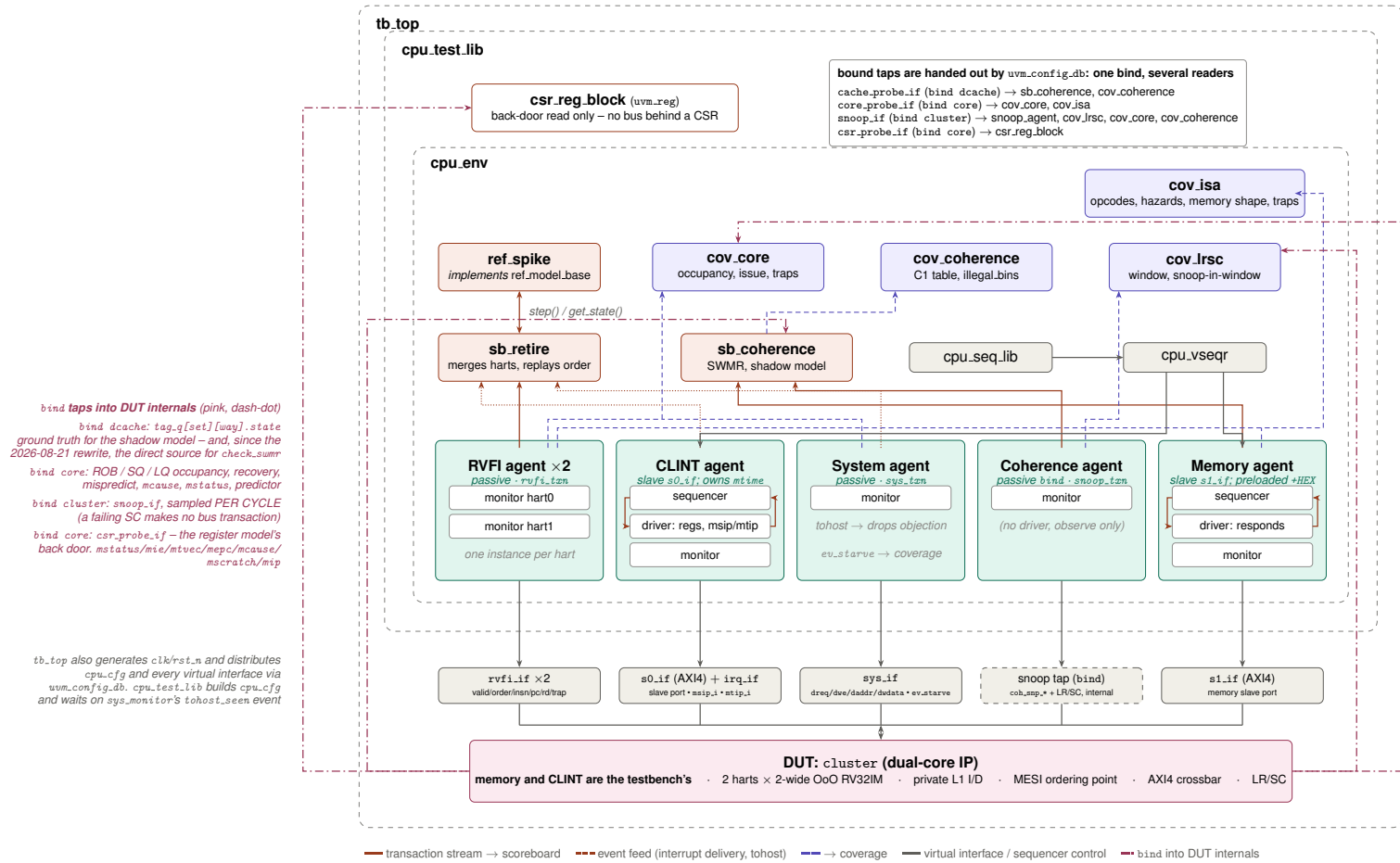


Figure 17: **The verification environment.** The design under test sits along the bottom. Above it, five agents connect to it through interfaces. Above them, two scoreboards decide whether what happened was allowed, and a reference model supplies the second opinion. Across the top, four coverage models record what was actually reached. Red lines carry observations to a scoreboard; purple dashes carry them to coverage; the pink dot-dash is a back-door read straight into the design's own registers.

The environment, piece by piece

Reading figure 17 from the bottom up.

The design under test is the whole two-core cluster: both cores, their caches, the coherence unit and the bus. Memory and the timer are supplied by the testbench rather than being part of the design, which is what lets the testbench control how slow memory is.

Five agents sit on it.

- Two **retirement agents**, one per core, both passive. They watch a dedicated set of wires that reports every instruction the core finishes, what it was, where it was, what it wrote, what it read. This is the single richest source of truth in the whole environment.
- A **timer agent**, active. It drives the timer and inter-core interrupt registers, so tests can make an interrupt arrive at a chosen moment.
- A **system agent**, passive. It watches for the program signalling that it has finished, and ends the test.
- A **coherence agent**, passive, watching the snoop channel between the two data caches.
- A **memory agent**, active. It answers the bus as memory would, with a configurable delay, which is how the latency sweeps in chapter 6 are produced.

Two scoreboards, deliberately not one.

The first replays the retirement stream. Every instruction either core finishes is fed to a reference model, and the result is compared field by field. Both cores are merged into a single ordered stream first, so that shared-memory programs are checked as one execution rather than two independent ones.

The second checks coherence directly. It keeps its own shadow copy of what each cache is allowed to be holding, updated from the bus traffic, and checks the protocol's central promise: that no two caches ever simultaneously believe they may write the same line.

They are kept separate because they are different oracles over different data, and merging them is a known way to end up with a checker that cannot fail for one of the two. There is an honest limit worth stating: the first scoreboard replays the order the design actually chose, so it can tell you the results were right *given that order*. It cannot tell you the order was allowed. That is the second scoreboard's job, and a separate set of ordering tests.

Four coverage models record what each run reached: instruction mix and hazards; core occupancy, issue behaviour and traps; the coherence transition table; and the reservation window used by atomic instructions.

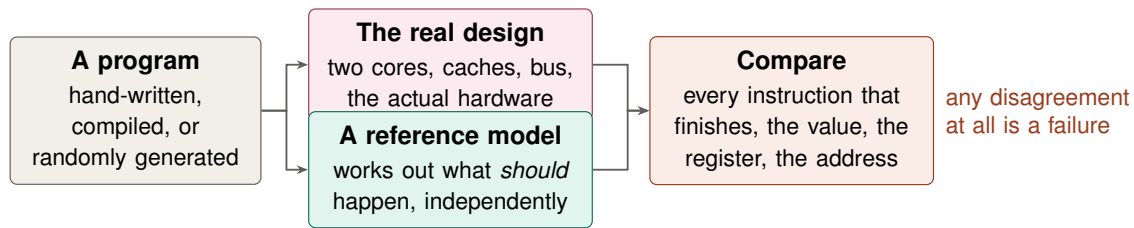
A **register model** sits off to the side. The machine's control registers are not reachable through any bus, by design, they are internal to the core, so the environment reads them through a back door directly into the design's own state, and compares against its own mirror.

from here on, how it was actually built

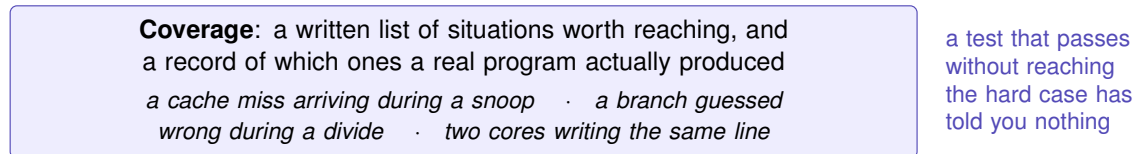
What the tests actually say

The battery. 91 automatic checks, all passing, none failing. Each one is a real run of the design, not a review. They cover unit tests of individual blocks, the full instruction-set compliance suite, the two-core coherence tests, and the environment above. The rule that made them worth having is that a change is not finished until every earlier number is unmoved.

1. Did it do the right thing?



2. But did the test actually reach anything interesting?



3. And would the checks have noticed if something were broken?

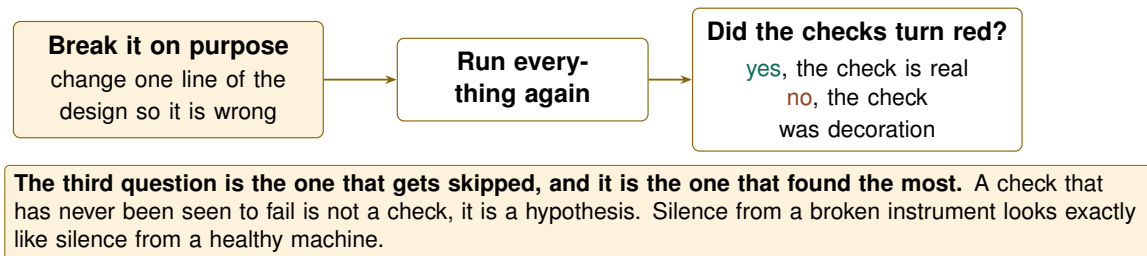


Figure 18: **Three questions, asked separately.** Correctness, reach, and whether the checks themselves work. The third is the one that gets skipped, and it is the one that found the most here.

Functional coverage: did the tests reach anything? A written list of 757 situations worth reaching, of which 636 were reached by a real program.

Those 757 are not a flat list. They are grouped into about seventy families across four models, and the families are worth seeing, because they say what the project decided was worth measuring:

- **Instruction level.** Every opcode. Operand shapes, including the signs of both inputs and whether they are the same register. Hazard distances between dependent instructions. Memory access shapes and alignments. Divide corners. Control-register accesses. Exception causes.
- **Microarchitectural.** Reorder buffer occupancy. How many instructions issued per cycle. Which unit caused a misprediction. Traps and their context. Recovery, including whether a checkpoint or the slow walk was used. Starvation.
- **Coherence.** Every cell of the protocol transition table, snoops crossed against the line state they landed on, and a cross of what both harts were holding at the same moment.
- **Atomics.** The distance between a reservation being made and used, every reason a reservation died, and what happens when a snoop lands in the middle of the window. This model exists because three of the hardest bugs in the project came from that one region, and nothing measured it.

A second kind of bin, which works in reverse. About thirty of these are **illegal bins**:

situations that must *never* occur. A cache line in two incompatible states at once, a reservation surviving something that should have killed it, a protocol transition the table does not contain. An ordinary bin failing to be hit is a gap in the testing. An illegal bin being hit is a failure of the design, reported immediately. The same instrument is therefore doing two opposite jobs at once.

The remaining 121 unhit bins are the interesting part. Each one carries a written proof of why it cannot happen, and a script checks those proofs against the measurement every time. Without that, “unreachable” quietly becomes “untested”. When those proofs were audited by hand, one of them turned out to be false, a situation claimed impossible that was in fact happening four times per run.

Random testing, and closing a hole without writing a test

Everything above is **directed** testing: a person decides what is worth exercising and writes a program that does it. It reaches what somebody thought of, and it stops there.

The complement is **constrained random** testing. Rather than writing programs, you write a *generator* and a set of constraints, and it produces hundreds of programs inside the distribution you described. It reaches things nobody thought of, in bulk, and it is bounded in a different way: it can only reach what its constraints allow.

The generator here takes knobs for program length, how often branches occur, how often loops occur and how many times they run, how much memory traffic there is, how many multiplies and divides, how many harts to emit code for, and how much of memory the two harts should *share* rather than keeping to private slices. Every generated program is still checked instruction by instruction against the reference model, exactly like a hand-written one.

One design rule in the generator is worth repeating because it is not obvious. **Termination is structural, not probabilistic.** A random program must never be able to hang, because the only thing that catches a hang is a timeout, and a timeout is indistinguishable from a dozen real defects. So loops are emitted with counted trip limits rather than with randomly generated exit conditions that are merely very likely to be met.

The loop that this exists for, run once, with its numbers. Eight random seeds, with no human targeting of any kind, reached **404 of the 757** situations. The thirty hand-written programs, which took a whole stage to write, reach 636. That the generator gets two thirds of the way there in an afternoon is the point of it.

Then the hole. Two coherence families were nearly empty: the protocol table at 8 cells of 62, and transitions at 12 of 32. The cause was not a missing program. It was a constraint: the sharing rate defaulted to zero, so each hart worked in its own private slice of memory and the two streams never once contended for the same line. A coherence protocol with nothing to be coherent about does nothing.

The fix was a constraint, not a program. Setting the sharing rate to 0.35 and re-running the same eight seeds:

	sharing 0.00	sharing 0.35
protocol transitions	12 of 32	23 of 32
protocol table cells	8 of 62	13 of 62
total bins reached	404 of 757	429 of 757

Still zero mismatches against the reference. Those bins did not close because somebody aimed a program at them. They closed because the generator was told to produce a different *distribution*, and then produced hundreds of programs inside it. That loop, generate, measure,

find the hole, change a constraint, re-measure, is the thing worth learning from this exercise, more than the numbers it produced.

And what it does not reach is also a result. Three families stay completely empty at any sharing rate, because the generator emits no atomic instructions at all, so it can never construct a reservation window. That is not a failure of random testing. It is random testing telling you precisely where directed tests are still doing the work, which is exactly the division of labour you want to be able to see.

Memory timing is randomised too, independently of the programs. Rather than running at one fixed memory latency, each run randomises the delay and the gap between transfers within a window, so the same program is tested against many different timings.

Code coverage: did the tests reach any of the design? Functional coverage only ever measures situations somebody thought to list. Code coverage measures something duller and complementary: which of the 2,082 individual lines and branches in the design were ever executed at all.

It earned its place immediately. It found that two thirds of the return-address stack had never run in the entire history of the project, because no test program had ever made a function call deep enough to use it. No list of interesting situations would have caught that, because nobody had thought to list it.

Mutation testing: would the tests have noticed? Break the design on purpose, in one specific place, and check that the tests turn red. 23 deliberate breakages so far: 18 caught, 4 that turned out to change nothing observable, and 1 missed, recorded as missed rather than explained away.

And the checkers themselves. The most useful thing this stage produced was not a bug in the processor. It was the discovery that half the checkers had never been shown capable of failing at all. They had been written, they ran, and they printed nothing, which looks exactly like working. Of 26 independent checks, 13 have now been demonstrated to fire when the thing they watch is broken.

That is an uncomfortable number to publish, and it is the right one, because the alternative is a green result nobody has earned. A checker that has never fired is a hypothesis, not a check.

What it cost: 91 checks green. 757 situations, every one either reached by a real program or carrying a machine-checked proof that it cannot be. 2,082 lines and branches tracked. And 13 of 26 checkers proven able to fail, the rest is honest, ordered, remaining work.

Chapter 10: What comes next

Everything below is known, written down, and deliberately not done yet. A project of this kind is never finished; it reaches a point where what is left is understood well enough to be listed.

Finish proving the checkers. Thirteen of twenty-six is not a stopping place. The work is not open-ended, each remaining one needs a specific deliberate breakage that makes it fire, and the list of which is already written.

Join up the two halves of the coherence logic. The protocol table exists once and is correct. But only one half of the machine actually goes through it: the half that answers other cores' questions. The half that asks its own questions computes the same decisions in a separate place. They agree today. Nothing in the design would notice if they stopped agreeing, and the right fix is a check that compares them rather than a test that hopes.

Add a second level of cache. Right now a miss goes straight to main memory. Chapter 6's measurement says exactly what that costs: 1,738 trips, each one paying the full latency.

Widen the route to memory. Chapter 8 identified the ceiling precisely. Both cores' data traffic funnels into a single bus master, and that is what limits the second core, not the coherence protocol. More masters, or a wider fabric, is the change that would move the number.

Let loads move ahead of stores more aggressively. Today a load waits until every older store has at least worked out its address. That is always safe and sometimes needlessly cautious. Letting loads go earlier, and detecting afterwards when that was wrong, is a well-understood next step, and the recovery machinery it needs already exists, built in chapter 7.

More than two cores. The core count is already a parameter. The bus underneath it is not: it is built for two masters and two destinations.

The thing worth keeping

If there is one habit this project would recommend, it is the one in chapter 9. It is easy to build something that works. It is much harder, and much more valuable, to build the thing that would tell you if it had stopped working, and then to check that *that* thing works too.

Nearly every serious error found here was of the same shape: something that looked correct because the instrument watching it was silent, and the instrument was silent because it was broken rather than because there was nothing to report.